RESEARCH PROPOSAL

Track B4 – BM25 Document Retrieval

Accelerating Full-Corpus BM25 Scoring with Custom CUDA Kernels

---

Group name:	*ChillBoy*
Members:	1. Le Ngoc Anh Khoa – 22127196
Topic:	B4 – BM25 Document Retrieval
Git repository:	https://github.com/pipoiwoczz/pga-bm25-gpu-project

---

1. Keywords

BM25 ranking, Information Retrieval, Inverted Index, Sparse Score Accumulation, GPU Parallel Search

2. List of References

Documentation, tutorials, technical reports

- NVIDIA CUB library documentation – parallel primitives for sort, scan, and reduce: https://nvlabs.github.io/cub/
- rank_bm25 Python package documentation (correctness reference implementation): https://github.com/dorianbrown/rank_bm25
- Elasticsearch BM25 similarity documentation (production reference for parameter conventions, k_1, b): https://www.elastic.co/guide/en/elasticsearch/reference/current/index-modules-similarity.html

Source code, pretrained models, demonstrative applications

- `rank_bm25` (Dorian Brown) – used as the CPU correctness oracle in `cpu_baseline.py`
- AG News dataset on Hugging Face (`fancyzhx/ag_news`) – real-text validation corpus (120K documents)
- MS MARCO passage ranking dataset (https://microsoft.github.io/msmarco/) – large-scale benchmark corpus (8.8M passages, 500K+ validation queries)
- Team's own repository: https://github.com/pipoiwoczz/pga-bm25-gpu-project (CPU baseline, already implemented and passing)

Papers

- Robertson, S. & Zaragoza, H. (2009). *The Probabilistic Relevance Framework: BM25 and Beyond.* Foundations and Trends in IR. https://www.staff.city.ac.uk/~sbrp622/papers/foundations_bm25_review.pdf
- Crane, M. et al. (2017). *Faster and Smaller Inverted Indices with Threading.* SIGIR 2017.
- Pibiri, G. & Trani, R. (2021). *Techniques for Inverted Index Compression.* ACM CSUR.

3. Content

3.1. Problem Statement

Problem: Full-corpus BM25 scoring for a batch of queries is serial and does not scale on CPU; posting-list traversal and score accumulation are embarrassingly parallel and map naturally onto GPU threads.

BM25 is the default ranking function behind production search engines such as Elasticsearch, Solr, and Lucene: given a batch of queries, it scores every document in a corpus by combining term frequency and inverse document frequency signals, then returns the top-K most relevant results. This is computationally expensive at scale – for each query term, the engine must traverse a **posting list** (the set of documents containing that term, which can range from a handful to millions of entries) and accumulate a weighted contribution into a running score for every one of those documents. On CPU this happens serially, one query term and one posting list at a time, which does not scale to corpora with hundreds of thousands to millions of documents scored against many queries per second. The workload is however a natural fit for the GPU: every (query term, document) pair in a posting list contributes independently to the final score, so posting-list traversal and score accumulation can be distributed across thousands of threads simultaneously. This project accelerates exactly that step – posting-list scoring and accumulation – while keeping index construction and final top-K extraction on CPU where they are already cheap and infrequent (index construction happens once, offline).

3.2. Dataset / Input

Dataset name and source:

- Development / correctness validation: **AG News** (Hugging Face, `fancyzhx/ag_news`) – 120,000 real news articles (title + description), 4 topics. Chosen because it is small to download (30 MB), requires no registration, and is real text rather than synthetic data.
- Full-scale benchmark: **MS MARCO Passage Ranking** – 8.8 million passages, loaded via `irds/msmarco-passage` on Hugging Face (`load_dataset("irds/msmarco-passage", "docs", streaming=True)`) which mirrors the official corpus without requiring registration or a bulk download. Besides, a **synthetic Zipfian corpus** generated in-house (already implemented in `utils.py`) is used when a controlled, arbitrarily large corpus size is needed to test GPU scaling behavior beyond what either real dataset provides.

Input size for benchmarking:

- Correctness / development runs: 50,000 – 120,000 documents, batches of 20 – 50 queries (2–5 terms each).
- Target benchmark scale (matches the catalog's performance target): **500,000 documents**, batch of **32 queries**.

How it will be loaded:

- AG News is loaded with the Hugging Face `datasets` library (`load_dataset("fancyzhx/ag_news")`) and flattened to `title + description` strings.
- MS MARCO is loaded via `load_dataset("irds/msmarco-passage", "docs", split="train", streaming=True)`, which streams passages one at a time without downloading the full 8.8 M-passage corpus upfront - avoiding both the registration requirement of the official mirror and the RAM pressure of a bulk load. I will collect up to 500 K passages for the largest benchmark run.
- The synthetic corpus is generated directly in NumPy with a Zipfian term-frequency distribution, reproducing the same posting-list-length skew (a few very common terms, a long tail of rare ones) found in real text, without any network download. It serves as a fallback if MS MARCO streaming is unavailable on Colab, and as a scaling probe beyond 500 K documents.
- Query terms for all three sources are sampled **proportional to document frequency** (not uniformly at random from the vocabulary), so that benchmark queries hit the long posting lists of common terms

rather than the near-empty lists of rare ones – the former is what drives real BM25 runtime and is the target of GPU acceleration.
- All three paths feed into the same tokenizer and inverted-index builder in `utils.py`, so nothing downstream (indexing, scoring, verification) changes between datasets.

Why GPU-suitable: The parallelism comes from two independent axes that the GPU implementation will exploit in stages:

1. **Within a posting list**: every `(document, term-frequency)` entry in a term's posting list can be scored independently - one thread (or warp) per posting, accumulated with an atomic add into a dense per-document score array. For a corpus of 500K documents, a single common term's posting list can contain tens of thousands of entries - all independent.
2. **Across queries**: a batch of 32 queries can be scored simultaneously by mapping queries to thread blocks and sharing the relevant inverted-index segments across the block via shared memory, rather than looping over queries serially as the CPU baseline does.

3.3. Background

BM25 (Best Match 25) is the standard probabilistic ranking function used by Elasticsearch, Solr, and Apache Lucene. Given a query q and a corpus of N documents, it assigns each document d a relevance score:

$$\text{score}(d, q) = \sum_{t \in q} \underbrace{\text{idf}(t)}_{\substack{\text{corpus-wide,} \\ \text{precomputed}}} \cdot \underbrace{\frac{f(t,d) \cdot (k_1 + 1)}{f(t,d) + k_1 \cdot \left(1 - b + b \cdot \frac{|d|}{\text{avgdl}}\right)}}_{\text{per-document TF saturation}}$$

where $f(t, d)$ is the raw term frequency of t in d, $|d|$ is document length, `avgdl` is the mean document length across the corpus, and $k_1 = 1.5$, $b = 0.75$ are standard tuning constants. The IDF component $\text{idf}(t) = \log(N - n_t + 0.5) - \log(n_t + 0.5)$ depends only on the corpus-level document frequency n_t and is computed once at index time.

Inverted index and posting lists. In practice, BM25 is not evaluated by iterating over all N documents for each query term – that would be $O(N \cdot |q|)$ per query. Instead, an offline **inverted index** maps each vocabulary term to a **posting list**: a sorted array of `(doc_id, term_frequency)` pairs for every document containing that term. At query time, only the posting lists of the query terms are touched:

Offline (once) – build inverted index:

```
index = {}
for doc_id, doc in enumerate(corpus):
  for term, tf in count_terms(doc).items():
    if term not in index:
      index[term] = []
      index[term].append((doc_id, tf))
```

Online (per query) – scatter-add over posting lists:

```
score_vector = zeros(N)                          # dense, length = corpus size
for term in query:
  idf = idf_table[term]
  for (doc_id, tf) in index[term]:             # posting list -- variable length!
    dl    = doc_len[doc_id]
    denom = tf + k1 * (1 - b + b * dl / avgdl)
    score_vector[doc_id] += idf * tf * (k1 + 1) / denom   # scatter-add
top_results = top_k(score_vector, k=10)
```

The inner loop – iterating a posting list and accumulating contributions into `score_vector` – is the **only** operation that scales with corpus size at query time. `cProfile` confirms this accounts for over 95% of runtime in the CPU baseline.

```
!python src/main.py --num-docs 500000 --num-queries 32 --profile

==================================================
  BM25 BENCHMARK RESULTS
==================================================
  Version:          cpu
  Dataset:          synthetic
  Corpus size:           500,000 documents
  Batch size:                 32 queries
  Index build time:     47855.81 ms
  Scoring time:            96.64 ms total
  Per-query:               3.020 ms/query
  Throughput:              331.1 queries/sec
  Correctness:      32/32 top-10 matched
  Target (V3 GPU):  500K docs, 32 queries < 100 ms
==================================================

Profiling (cProfile, custom scorer):

         577 function calls in 0.089 seconds

   Ordered by: cumulative time

   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
       32    0.051    0.002    0.089    0.003 /content/pga-bm25-gpu-project/src/
cpu_baseline.py:67(score)
      256    0.030    0.000    0.030    0.000 {method 'at' of 'numpy.ufunc' objects}
       32    0.007    0.000    0.007    0.000 {built-in method numpy.zeros}
      256    0.000    0.000    0.000    0.000 {method 'get' of 'dict' objects}
        1    0.000    0.000    0.000    0.000 {method 'disable' of '_lsprof.Profiler'
objects}
```

Where the parallelism lives. The scatter-add has two independent axes that the GPU will exploit:

1. **Within one posting list:** every `(doc_id, tf)` entry contributes to a **different** element of `score_vector` (or at worst to the same element, handled by `atomicAdd`). There are no data dependencies between entries – they can all be computed simultaneously by parallel threads.
2. **Across query terms:** the posting lists of different query terms are completely independent of each other. A query with 8 terms produces 8 independent scatter-add passes over disjoint (or overlapping but independently accumulating) regions of `score_vector`. These can be parallelized across thread blocks.
3. **Across queries in a batch:** a batch of 32 queries each needs its own `score_vector`, but they all read from the **same** inverted index. Threads processing different queries can share index data loaded into shared memory, amortizing the cost of reading from global memory.

3.4. The Challenge

Unlike dense workloads (matrix multiply, convolution), BM25 is an **irregular, sparse** problem with three structural challenges:

- **Load imbalance:** posting-list lengths follow a Zipfian distribution – a handful of common terms have lists orders of magnitude longer than rare ones, causing severe thread divergence under a naive one-thread-per-term mapping.
- **Atomic contention:** multiple query terms can target the same document slot, serializing concurrent `atomicAdd` writes without shared-memory buffering.
- **Irregular memory access:** posting lists sit in disjoint, variable-length memory regions with no regular stride, making coalesced access non-trivial.

These three properties are what the V1 → V2 → V3 stages address in sequence, and what we hope to learn by doing this project.

What we hope to learn: how to design and tune GPU kernels for **irregular, data-dependent** workloads – a very different discipline from the dense, uniform-stride kernels covered in the image-processing tracks, and directly transferable to how real search engines (and sparse ML workloads more broadly) are accelerated in production.

3.5. Risk Analysis

Risk	Description	Mitigation
Colab T4 resource limits	Free-tier Colab has limited RAM/GPU memory; earlier tests already hit OOM at 500K synthetic documents (3.9 GB limit).	Validate kernels at reduced scale (100K–200K docs) first, use `int32` arrays, stream data in chunks, keep the synthetic corpus as fallback.
Single-member team	One-person team with no backup if a given week hits an unexpected blocker (e.g. a hard-to-debug CUDA bug).	Treat GPU V1–V3 as the committed 100% deliverable; GPU V4 is an optional stretch goal, dropped first if behind schedule.
Dataset access (MS MARCO)	The official MS MARCO corpus requires registration and a bulk download – it cannot be fetched directly from a URL.	Load it via Hugging Face `datasets` (`irds/msmarco-passage`, `streaming=True`) instead of the official mirror; fall back to the synthetic Zipfian corpus if HF streaming is also unavailable.

3.6. Resources

Starting point: I am starting from an **existing, already-working CPU baseline** (not from scratch for the CPU portion): `src/cpu_baseline.py` implements and profiles both the `rank_bm25` reference implementation and a hand-written NumPy inverted-index scorer, `tests/test_correctness.py` verifies exact top-10 agreement between the two, and `cProfile` output has already confirmed the posting-list scoring step as the bottleneck. The GPU kernels (V1–V4) will be written from scratch in Numba CUDA / CuPy on top of this baseline.

Compute: Google Colab with a free-tier **T4 GPU** runtime. This is sufficient for the corpus sizes targeted in this proposal (up to 500K documents / 32-query batches) and for iterating on Numba CUDA kernels without requiring dedicated lab hardware. If profiling later shows the T4's memory bandwidth is a limiting factor at the largest benchmark scale, I will evaluate moving later benchmark runs to a lab-provided GPU.

Reference material: NVIDIA CUB library (for the optional V4 top-K step), the BM25 paper (Robertson & Zaragoza 2009) for the exact scoring formula and IDF conventions, and the `rank_bm25` source for correctness parity (already reverse-engineered and matched exactly in the CPU baseline, including its epsilon-floor handling of negative IDF values).

Datasets: AG News (Hugging Face `fancyzhx/ag_news`) for correctness validation; MS MARCO Passage Ranking for the large-scale benchmark, loaded via Hugging Face `irds/msmarco-passage` with `streaming=True` since the official corpus requires registration and a bulk download.

3.7. Optimization Plan

Stage	Technique	Targets	Expected outcome
CPU baseline	`rank_bm25` reference + hand-written NumPy inverted index	Correctness oracle + timing baseline	Done – 32/32 top-10 match, bottleneck confirmed via `cProfile`
GPU V1	One thread block per query term; naive `atomicAdd` accumulation	Functional parallel port	Correct results; some speedup, but limited by load imbalance + atomic contention
GPU V2	Warp-level reduction before the atomic write; shared-memory score buffer per block	Load imbalance, atomic contention	Meaningful speedup jump as contention drops
GPU V3	Batch all 32 queries simultaneously; shared inverted-index segments across queries in a block	GPU under-utilization between queries	Reach 15–50x target range
GPU V4 (stretch)	CUB `DeviceRadixSort` for parallel top-K, replacing the CPU top-K step	End-to-end GPU pipeline	Full pipeline under 100 ms, no CPU round-trip for ranking

3.8. Division of Work

This project is completed by a single member. All implementation, benchmarking, and reporting tasks are handled by the same person.

Le Ngoc Anh Khoa	CPU baseline, inverted index, correctness tests (done); GPU V1–V3 kernels; benchmark harness; dataset pipeline; report and proposal writing.

75%	• GPU V1 (naive posting-list kernel, one thread block per query term, atomic accumulation) implemented and functionally correct: top-10 results match the CPU reference exactly on corpora up to 100K documents. • Any measurable speedup over the CPU scoring step is achieved and reported – the point of this tier is a **working, verified** GPU kernel, not a specific number.
100%	• GPU V1, V2 (warp reduction + shared-memory score buffer), and V3 (32-query batching with shared inverted-index segments) all implemented, correct, and benchmarked. • Achieve **at least 10-15x speedup** over the CPU posting-list scoring step at 500K documents / 32 queries batch. • End-to-end correctness maintained: top-10 retrieved documents match the CPU reference exactly across the full benchmark query batch. • Reported queries-per-second throughput at 100K and 500K document corpus sizes.

125%	• GPU V4: parallel top-K selection via CUB `DeviceRadixSort`, completing an end-to-end GPU pipeline. • Score 500K documents for a 32-query batch in **under 100 ms**, toward the upper end (50x per-query) of the catalog's target speedup range. • Benchmark across two corpus scales (100K and 1M documents) to characterize how speedup and load-imbalance behavior change with scale.

Demo plan: a live query in a CLI/notebook showing (a) top-10 results from the CPU reference and the GPU kernel side-by-side to demonstrate correctness, and (b) a bar chart of queries-per-second for CPU vs. GPU V1/V2/V3(/V4) at increasing corpus sizes, illustrating where each optimization stage's gains come from.

4. Weekly Schedule

Week	Le Ngoc Anh Khoa
Week 01 (29/06 - 03/07)	Select topic B4; read Robertson & Zaragoza (2009); initialize repository; survey `rank_bm25` source and AG News / MS MARCO datasets.
Week 02 (04/07 - 10/07)	*(done)* Implement `cpu_baseline.py`: `rank_bm25` reference + `NumpyBM25` inverted index. Fix IDF formula to match `rank_bm25` exactly (ATIRE variant). Write `tests/test_correctness.py`; verify 32/32 top-10 match. Profile with `cProfile`, confirm `NumpyBM25.score` as bottleneck. Write `README.md`.
Week 03 (11/07 - 17/07)	Finalize and submit proposal; set up Colab T4 environment; implement dataset loaders (AG News / MS MARCO / synthetic); begin GPU V1 (Numba CUDA): naive posting-list kernel + `atomicAdd`.
Week 04 (18/07 - 24/07)	Complete GPU V1; verify correctness against CPU reference; begin GPU V2 (warp-level reduction + shared-memory score buffer); build benchmark harness (queries/sec, latency by corpus size).
Week 05 (25/07 - 31/07)	Complete GPU V2; benchmark V1 vs V2 to measure atomic-contention fix; begin GPU V3 (32-query batching, shared inverted-index segments); run benchmark suite at 100K / 500K documents.
Week 06 (01/08 - 07/08)	Complete GPU V3; run full benchmark suite; verify end-to-end correctness; GPU V4 (stretch: CUB `DeviceRadixSort`); prepare demo (live query + speedup chart); write final report.